



Instituto Tecnológico de Aeronáutica - ITA
CM-204 - Aprendizado Profundo
Aluno: Ulisses Lopes da Silva

Relatório do Laboratório 04 - Segmentação Semântica com U-Net

1 Breve Explicação em Alto Nível da Implementação

Foram implementadas duas partes do código-base fornecido: a definição das camadas da arquitetura U-Net e a função de avaliação do modelo.

1.1 Arquitetura U-Net

A U-Net é composta por um *encoder*, um *bottleneck* e um *decoder* simétrico. Cada bloco convolucional utiliza o módulo auxiliar `DoubleConv` – já fornecido – que aplica duas convoluções 3×3 com ReLU e dropout entre elas.

No *encoder*, o número de filtros dobra a cada nível ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$), com camadas `MaxPool2d` de stride 2 reduzindo a resolução espacial pela metade a cada etapa. O *bottleneck* aplica um `DoubleConv` com 1024 filtros. No *decoder*, camadas `ConvTranspose2d` (stride 2) restauram a resolução, reduzindo os filtros pela metade ($512 \rightarrow 256 \rightarrow 128 \rightarrow 64$); cada *upsampling* é seguido de uma concatenação com o mapa de *features* correspondente do *encoder* (*skip connection*) e de um `DoubleConv` cujas entradas têm o dobro dos filtros esperados na saída, por causa da concatenação. A camada de saída é uma `Conv2d` 1×1 que mapeia os 64 canais para as 6 classes.

```
# Encoder
self.conv1 = DoubleConv(in_channels, 64)
self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
self.conv2 = DoubleConv(64, 128)
self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
self.conv3 = DoubleConv(128, 256)
self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2)
self.conv4 = DoubleConv(256, 512)
self.pool4 = nn.MaxPool2d(kernel_size=2, stride=2)

# Bottleneck
self.conv5 = DoubleConv(512, 1024)

# Decoder
self.upconv6 = nn.ConvTranspose2d(1024, 512, kernel_size=2, stride=2)
self.conv6 = DoubleConv(1024, 512) # 512 (up) + 512 (skip) = 1024 in
```

```

self.upconv7 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
self.conv7 = DoubleConv(512, 256)
self.upconv8 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
self.conv8 = DoubleConv(256, 128)
self.upconv9 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2)
self.conv9 = DoubleConv(128, 64)

```

Listing 1: Definição das camadas da U-Net (construtor).

1.2 Função de Avaliação (evaluate_unet)

A função coloca o modelo em modo de avaliação (`model.eval()`) e desabilita o cálculo de gradientes (`torch.no_grad()`). Em seguida, itera sobre cada um dos três `DataLoaders`, acumulando a perda *batch a batch* e calculando a média ao final. O padrão é idêntico ao laço de validação do código de treinamento fornecido na Seção 4.4 do *notebook*.

```

model.eval()
with torch.no_grad():
    # Treino
    train_loss = 0.0
    for images, masks in train_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        train_loss += criterion(outputs, masks).item()
    train_loss /= len(train_loader)

    # Validacao
    val_loss = 0.0
    for images, masks in val_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        val_loss += criterion(outputs, masks).item()
    val_loss /= len(val_loader)

    # Teste
    test_loss = 0.0
    for images, masks in test_loader:
        images, masks = images.to(device), masks.to(device)
        outputs = model(images)
        test_loss += criterion(outputs, masks).item()
    test_loss /= len(test_loader)

    return (train_loss, val_loss, test_loss)

```

Listing 2: Implementação de `evaluate_unet`.

2 Figuras Comprovando Funcionamento do Código

2.1 Sumário do Modelo (torchinfo)

A Tabela 1 resume a arquitetura da rede. O total de parâmetros treináveis é **31.032.070**, conforme esperado.

Tabela 1: Sumário simplificado gerado por `torchinfo.summary(model, input_size=(1,3,256,256))`.

Camada	Shape de saída	Parâmetros
UNet	[1, 6, 256, 256]	—
DoubleConv – encoder 1	[1, 64, 256, 256]	38.720
MaxPool2d	[1, 64, 128, 128]	—
DoubleConv – encoder 2	[1, 128, 128, 128]	221.440
MaxPool2d	[1, 128, 64, 64]	—
DoubleConv – encoder 3	[1, 256, 64, 64]	885.248
MaxPool2d	[1, 256, 32, 32]	—
DoubleConv – encoder 4	[1, 512, 32, 32]	3.539.968
MaxPool2d	[1, 512, 16, 16]	—
DoubleConv – bottleneck	[1, 1024, 16, 16]	14.157.824
ConvTranspose2d	[1, 512, 32, 32]	2.097.664
DoubleConv – decoder 1	[1, 512, 32, 32]	7.078.912
ConvTranspose2d	[1, 256, 64, 64]	524.544
DoubleConv – decoder 2	[1, 256, 64, 64]	1.769.984
ConvTranspose2d	[1, 128, 128, 128]	131.200
DoubleConv – decoder 3	[1, 128, 128, 128]	442.624
ConvTranspose2d	[1, 64, 256, 256]	32.832
DoubleConv – decoder 4	[1, 64, 256, 256]	110.720
Conv2d 1 × 1 (saída)	[1, 6, 256, 256]	390
Total		31.032.070

2.2 Predições no Conjunto de Teste

As Figuras 1–4 mostram, da esquerda para a direita: a imagem de entrada, a máscara verdadeira (*ground truth*) e a máscara predita pelo modelo.

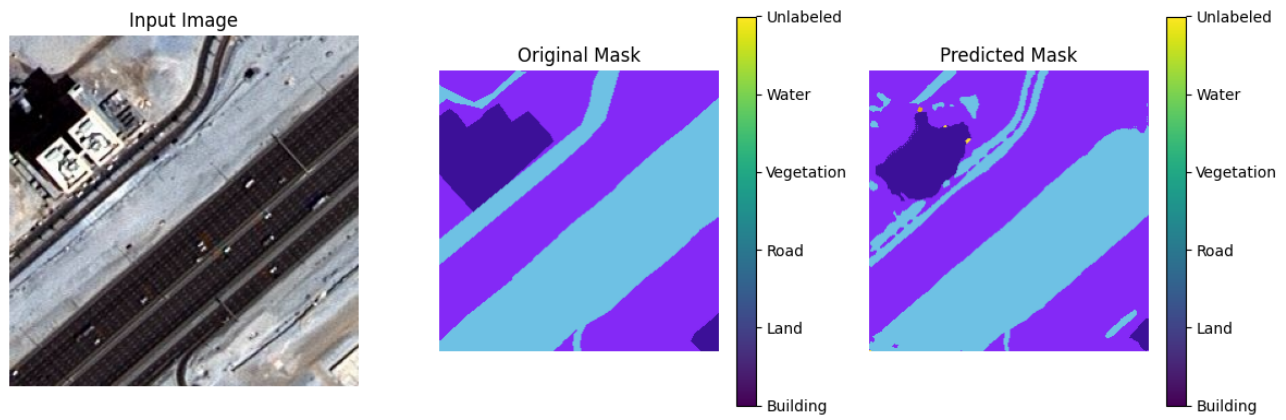


Figura 1: Imagem de teste 0: área com estrada e construção.

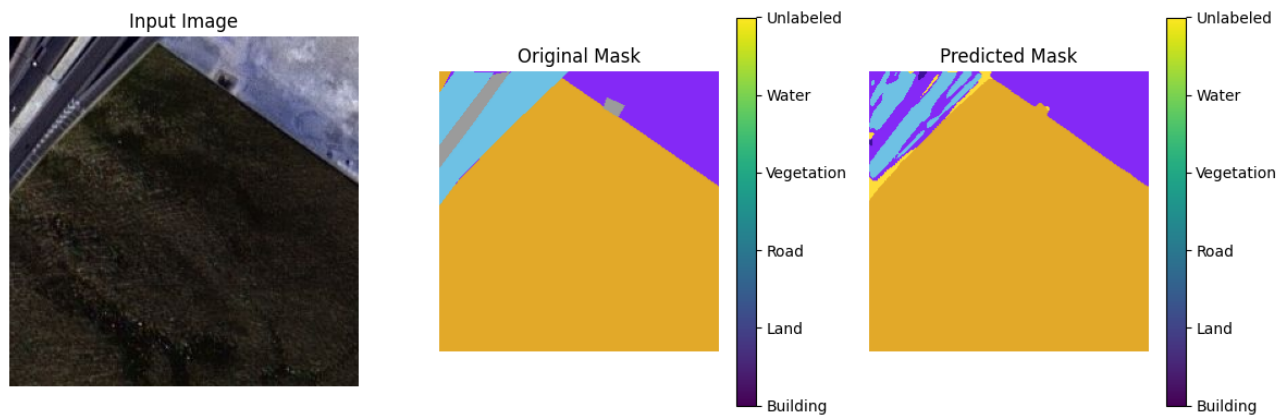


Figura 2: Imagem de teste 1: área de vegetação com estrada.

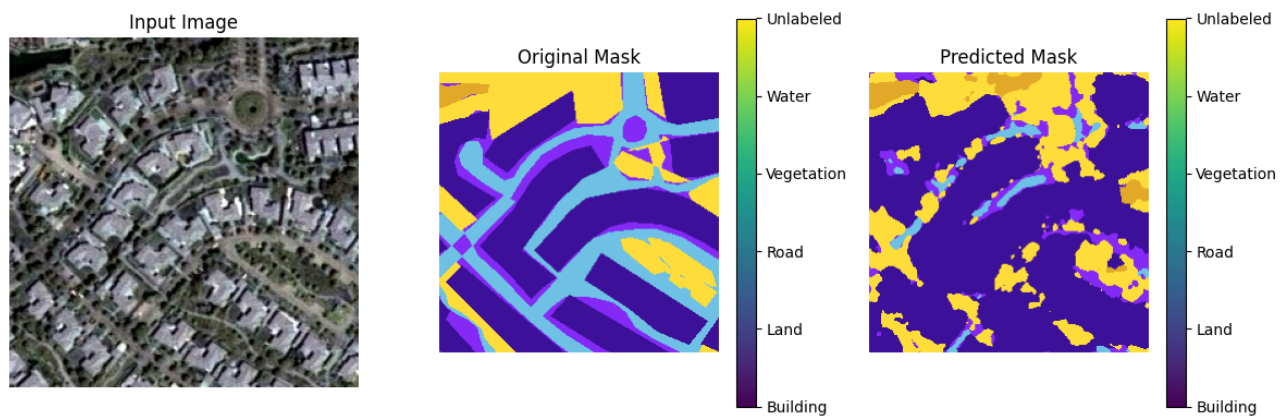


Figura 3: Imagem de teste 2: área urbana densa com múltiplas classes.

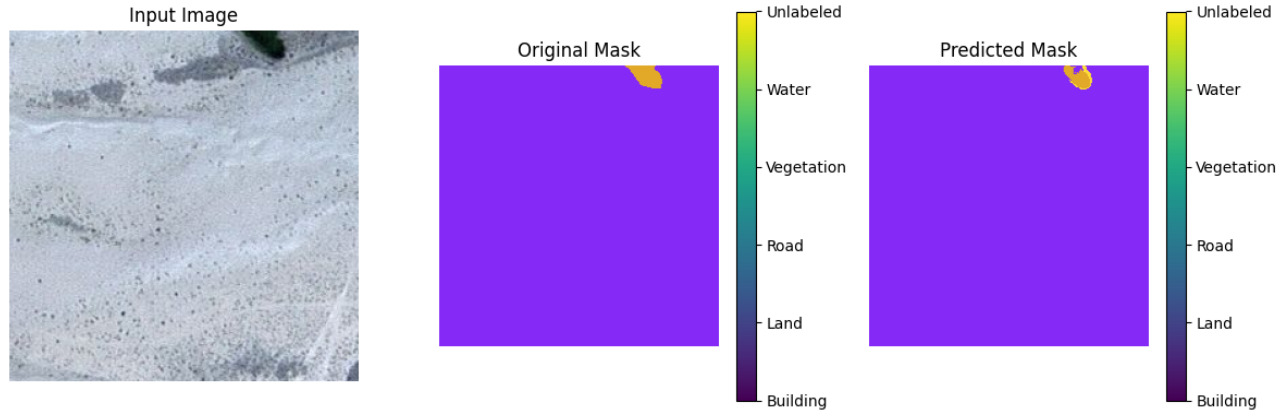


Figura 4: Imagem de teste 3: área homogênea de terra/areia.

3 Discussão dos Resultados

3.1 Perdas Obtidas

A Tabela 2 apresenta os valores de perda (*Cross-Entropy*) nos três conjuntos após o carregamento dos pesos pré-treinados.

Tabela 2: Perdas médias por conjunto.

Métrica	Treino	Validação	Teste
Perda (<i>Cross-Entropy</i>)	0,4390	0,4022	0,4141

3.2 Análise do *Overfitting*

O modelo apresenta perda de treino (0,4390) ligeiramente superior às perdas de validação (0,4022) e teste (0,4141). Esse comportamento, aparentemente contraintuitivo, é consistente com o uso de *early stopping*: os pesos salvos correspondem à época de menor perda no conjunto de validação (época 29, dentre 80 treinadas). Como o modelo escolhido é aquele que melhor generalizou durante o treinamento, e não o do final do treinamento onde a rede estaria mais ajustada ao treino, a perda de treino pode ser maior do que a de validação.

Adicionalmente, o *dropout* presente no **DoubleConv** é ativo apenas durante o treinamento, elevando artificialmente a perda observada no conjunto de treino em comparação com a avaliação (que o desativa). Esse efeito é esperado e não indica problema no modelo.

Em suma, os resultados são coerentes com um modelo sem *overfitting* severo: as perdas de validação e teste estão próximas entre si (diferença de $\approx 0,012$) e ambas menores que a perda de treino. É válido ressaltar, também, que, conforme o gráfico mostrado ao final do *notebook*, após a época 29, o modelo claramente entra em *overfitting*. Nesse sentido, o *early stopping* também ajudou a prevenir esse fenômeno no modelo geral.

3.3 Análise Qualitativa das Predições

Observando as Figuras 1–4, nota-se que o modelo acerta bem regiões de grande extensão e textura uniforme, como vegetação (Figura 2) e terra (Figura 4). Em imagens com maior diversidade de classes (Figuras 1 e 3), o modelo captura os grandes padrões, mas comete erros nos contornos e em classes menos representadas – como *Road* e *Building* – o que é esperado dada a complexidade dessas regiões em imagens aéreas.